

# LangChain vs LangGraph

*Interview-Focused Notes — Definitions, Easy Explanations, and Flow Diagrams*

---

## Introduction

LangChain is the component library that makes it easy to build LLM-powered features. LangGraph is the orchestration framework built on top of LangChain that lets you coordinate those features into complex, stateful, production-grade workflows.

This document walks through why LangGraph was created, using the six real limitations of LangChain that show up once workflows stop being simple and linear. Each topic includes a definition, a plain-English explanation, and a diagram, followed by interview-style questions at the end.

## Part 1 — Quick LangChain Recap

**Definition:** LangChain is an open-source library designed to simplify the process of building LLM-based applications, by providing modular building blocks.

Easy explanation: Building an LLM app from scratch means wiring together a model, a prompt, maybe a document search system, and connecting their outputs to inputs. LangChain gives you ready-made pieces for each of these jobs, so you just plug them together instead of writing everything from zero.

### Four Core Components

- **Model** — a unified interface to call any LLM provider (OpenAI, Anthropic, Hugging Face, Ollama) with minimal code changes.
- **Prompts** — templates and tools for structured, reusable prompt engineering.
- **Retrievers** — fetch relevant documents from a vector store or knowledge base (the foundation of RAG apps).
- **Chains** — link components together so one block's output automatically becomes the next block's input. This is LangChain's biggest offering.



Figure 1: The four core LangChain components.

**What you can build with it:** simple chatbots, multistep report generation, RAG applications, and basic tool-using agents — anything that is mostly a straight line from input to output.

## Part 2 — Workflow vs Agentic AI

Before comparing frameworks, it helps to separate two ideas that are often confused: a workflow and an agentic AI system.

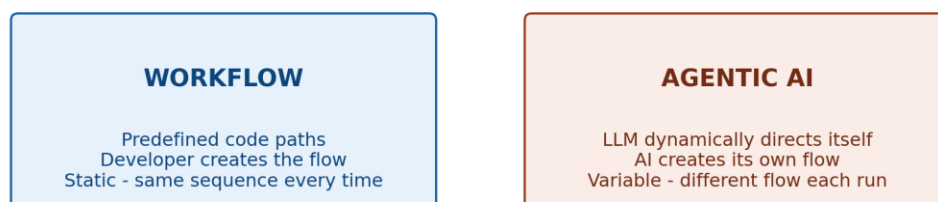


Figure 2: A workflow follows a path the developer designed; an agent designs its own path.

**Workflow:** LLMs and tools are orchestrated through predefined code paths. The developer creates the entire flow structure, and it executes the same way every time.

**Agentic AI:** the LLM dynamically directs its own process and tool usage, planning and adapting its path based on context — the flow can be different on every run.

Why it matters: an "automated hiring system" that always follows the same fixed steps (create JD, post, wait, shortlist, interview, onboard) is a workflow, not an agent — even though it looks intelligent and uses LLMs at several points. LangGraph is built to handle both — complex workflows and true agentic behavior.

## Part 3 — Six Key Challenges: Why LangGraph Was Needed

As soon as a workflow becomes non-linear (branches, loops, waiting periods, multiple tools, human approvals), LangChain starts to struggle. These are the six recurring problems that LangGraph was built to solve.

### Challenge 1 — Control Flow Complexity

**Definition:** The ability of a workflow to support branching (if/else decisions), looping (retry logic), and jumps between steps, instead of a single straight line.

Easy explanation: A simple LangChain chain is like a straight road — step 1 always leads to step 2, then step 3. Real workflows need traffic signals and U-turns: "if approved, go here, else go back and retry." LangChain has no native concept of branches or loops in its chain structure, so this needs custom Python glue code. LangGraph represents the workflow as a graph, where branching and looping are built-in, native features.

| Aspect            | LangChain                 | LangGraph                      |
|-------------------|---------------------------|--------------------------------|
| Flow structure    | Linear chains only        | Graph — branches, loops, jumps |
| Adding a decision | Requires custom glue code | A conditional edge             |
| Retry logic       | Manual while-loops        | Native loop back to a node     |

### Challenge 2 — State Handling

**Definition:** State is the collection of important data points (like a job description, application count, or approval status) that must be tracked and carried forward as a workflow executes.

Easy explanation: LangChain is stateless by design — it only offers conversational memory (chat history), not general-purpose data storage across steps. So developers must build and manage a global dictionary by hand, remembering to read and update it at every single step. That's slow and very easy to get wrong. LangGraph is stateful by design — every node automatically receives the current state and returns an updated version, and the framework takes care of passing it forward.

### LangChain - Manual State (Stateless)



Every step must remember to read and write the global dictionary by hand

### LangGraph - Automatic State (Stateful)



The framework passes and updates state automatically between nodes

Figure 3: LangChain needs manual state tracking; LangGraph passes state automatically.

| Aspect         | LangChain                        | LangGraph                         |
|----------------|----------------------------------|-----------------------------------|
| Architecture   | Stateless                        | Stateful                          |
| State storage  | Manual global dictionary         | Built-in state object (TypedDict) |
| Updating state | Manual modification, error-prone | Return-based, framework managed   |

## Challenge 3 — Event-Driven Execution

**Definition:** The ability of a workflow to pause at a specific point, wait for an external trigger (a timer, a reply, an API callback), and then resume exactly where it left off.

Easy explanation: LangChain chains run start to finish without stopping — they were never designed to pause. So waiting 7 days for job applications, or waiting indefinitely for a candidate's offer response, requires breaking the workflow into separate chains, manually saving and loading state, and gluing them together with external timers or cron jobs. LangGraph has built-in checkpoints: it can save the current state, pause for any length of time, and resume from that exact point automatically when the trigger event arrives.

#### LangChain - Sequential Only



Cannot pause for 7 days - needs external cron jobs and manual state save/load (glue code)

#### LangGraph - Event-Driven (Checkpoints)



Workflow pauses, waits for an external trigger, then resumes from the exact point - no glue code

Figure 4: LangChain cannot pause mid-execution; LangGraph checkpoints and resumes natively.

### Challenge 4 — Fault Tolerance

**Definition:** The ability to recover from a failure without starting the entire workflow over from the beginning.

Easy explanation: If a LangChain chain crashes on step 7 of 10, there's no built-in way to resume from step 7 — you typically have to restart from step 1 and rebuild the missing context manually. LangGraph's checkpoint system means a crash can be resumed from the last saved state, so a failure in a long workflow doesn't waste all the earlier work.

**Real example:** In the hiring workflow, if the system crashes right after posting the job, LangGraph can resume from "job posted, now monitor applications" instead of recreating and reposting the job description from scratch.

### Challenge 5 — Human-in-the-Loop

**Definition:** The workflow's ability to pause and wait for a human decision — sometimes for an extended period — before continuing.

Easy explanation: LangChain supports only very basic approval-style interactions within a single running chain. It cannot cleanly pause a workflow for hours or days while waiting on a human (e.g. a manager approving a job description, or a candidate accepting an offer). LangGraph treats a human decision the same way it treats any external trigger — the workflow pauses via a checkpoint and resumes automatically once the human responds, no matter how long that takes.

**Real example:** Sending an offer letter and waiting indefinitely for the candidate to accept or reject it is a natural pause point that LangGraph handles natively.

### Challenge 6 — Observability

**Definition:** How easily you can monitor, debug, and understand exactly what a workflow did at runtime — including custom logic, not just LLM calls.

Easy explanation: LangSmith (LangChain's monitoring tool) can only track the parts of a workflow that are actual LangChain calls. Any custom Python "glue code" you wrote to handle loops, state, or waiting — remains invisible to it. So you can see the LLM calls, but not the full picture of why a decision was made. Because LangGraph has tight, native LangSmith integration, and nodes/edges/state changes are all part of the framework itself (not glue code), everything gets tracked automatically — node entry/exit, state before and after, timing, and human interactions.

**Real example:** An agent that keeps posting job ads and racks up a huge bill — with LangChain + glue code you may never find out why it kept looping; with LangGraph the complete decision timeline is available for audit.

## Part 4 — Bonus Feature: Nested Workflows (Subgraphs)

**Definition:** A subgraph is a graph that is used as a single node inside another graph — in other words, a workflow within a workflow.

Easy explanation: Some steps that look simple on a high-level flowchart (like "Conduct Interview") are actually complex processes in their own right (technical round, behavioral round, cultural fit round, final evaluation). Instead of cluttering the main workflow with all that detail, LangGraph lets you build that detail as its own graph and drop it into the main workflow as one node. This keeps the main flow readable while still modeling the full complexity underneath.

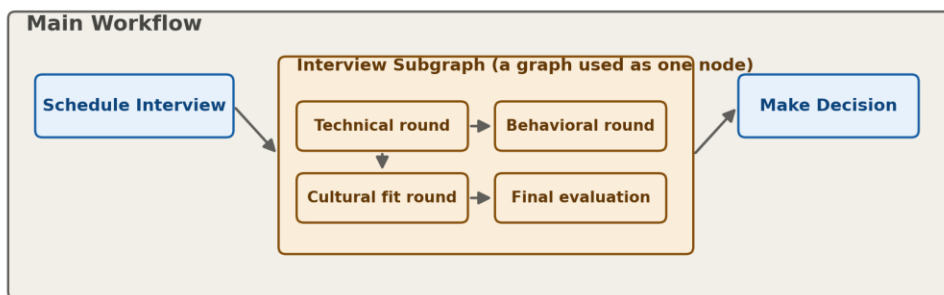


Figure 5: The interview subgraph is a full graph that acts as a single node in the main workflow.

### Two Major Use Cases

- Multi-agent systems — a central control agent orchestrates specialized sub-agents (e.g. a self-driving car's driving agent, sensor agent, and entertainment agent), each potentially a subgraph.

- Reusability — the same subgraph (e.g. an approval workflow) can be reused at multiple points in the main graph, following the DRY (Don't Repeat Yourself) principle.

**LangChain limitation:** if a single complex workflow is already difficult to build in LangChain, nesting one workflow inside another is effectively not possible without an explosion of custom glue code.

## Part 5 — Should You Abandon LangChain?

**Answer:** No. LangGraph is built on top of LangChain, not a replacement for it.

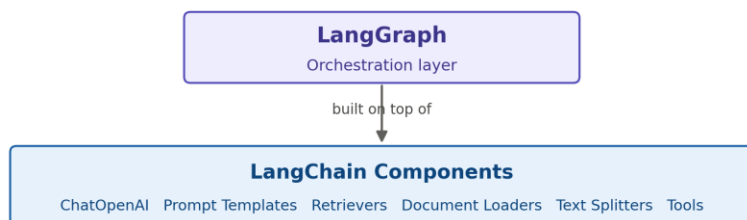


Figure 6: LangGraph is the orchestration layer sitting on top of LangChain's components.

Easy explanation: LangChain still provides the individual building blocks — LLM integrations, prompt templates, retrievers, document loaders, text splitters, and tools. LangGraph adds the orchestration layer on top: complex flow control, state management, event-driven pausing, and fault tolerance. In real projects you typically use LangChain components inside LangGraph nodes — you don't choose one over the other, you combine them.

**Division of labor:** LangChain says "here are the tools you need"; LangGraph says "here's how to coordinate them in complex ways."

### Decision Framework

| Use LangChain When...                  | Use LangGraph When...                         |
|----------------------------------------|-----------------------------------------------|
| Simple, linear input to output tasks   | Complex, non-linear, branching workflows      |
| Basic chatbots, summarizers, RAG apps  | Multi-agent systems and orchestration         |
| Minimal state, short-running processes | Rich state management, long-running processes |
| Quick prototypes / demo applications   | Production-grade, enterprise applications     |
| No need to pause or wait for events    | Human-in-the-loop and event-driven pauses     |



## Part 6 — Interview Questions (Simple, Not Bookish)

Phrased the way an interviewer would actually ask — good for quick revision before an interview.

### **Q1. In one line, what's the real difference between LangChain and LangGraph?**

LangChain gives you the building blocks for LLM apps (models, prompts, retrievers, chains);  
LangGraph gives you a way to orchestrate those blocks into complex, stateful, pausable workflows.

### **Q2. Why do people say LangChain is "stateless"?**

Because out of the box it only offers conversational memory (chat history) — there's no built-in mechanism to store and pass arbitrary workflow data (like scores, statuses, counts) between steps. You'd have to build and manage that yourself with a global dictionary.

### **Q3. What's the difference between a workflow and an agentic AI system?**

A workflow follows a path the developer designed in advance and runs the same way every time. An agent decides its own path dynamically at runtime, so the flow can differ from run to run.

### **Q4. Give a concrete example of why LangChain can't handle event-driven execution well.**

If a job needs to sit on LinkedIn for 7 days before checking applications, a LangChain chain can't just pause — you'd need to split it into two chains, save state manually, use an external timer/cron job to wait, then reload the state into the second chain. LangGraph handles this pause and resume natively with checkpoints.

### **Q5. Why is LangGraph's observability better than plain LangChain + LangSmith?**

LangSmith only tracks actual LangChain calls. Any custom Python code you wrote for loops, retries, or state handling is invisible to it. In LangGraph, nodes, edges, and state changes are part of the framework itself, so LangSmith can see the complete picture — not just the LLM calls.

### **Q6. What is a subgraph, and why would you use one?**

A subgraph is a full graph used as a single node inside a bigger graph — a workflow within a workflow. You'd use it to keep a complex sub-process (like a multi-round interview) readable inside the main flow, and to reuse the same sub-process (like an approval flow) in multiple places without duplicating logic.

### **Q7. Is LangGraph a replacement for LangChain? Why or why not?**

No — LangGraph is built on top of LangChain. LangChain still provides the individual components (models, prompts, retrievers, tools); LangGraph adds orchestration on top: state, branching, loops, pausing, and fault tolerance. Most real projects use both together.

### **Q8. If someone gives you a simple Q&A chatbot requirement, would you reach for LangGraph? Why or why not?**

Probably not by default — a simple, linear, short-running chatbot is exactly what LangChain handles well on its own. Reaching for LangGraph there would be added complexity without a real benefit. LangGraph earns its place once you need branching, state, long waits, or human approval.

#### Q9. What real business problem does fault tolerance solve in a hiring workflow?

If the system crashes after the job is posted, LangChain would typically have to restart from scratch — recreating and reposting the job description. LangGraph's checkpointing lets it resume from "job already posted, now waiting for applications," avoiding duplicate work and wasted API calls.

#### Q10. Why does control flow complexity matter even for something that looks simple, like a hiring workflow?

Because real workflows are rarely a straight line — there's an approval loop for the JD, a retry loop if not enough applications come in, and a rejection/acceptance branch for each candidate. LangChain has no native way to express branches and loops, so all of that becomes hand-written glue code, which is exactly the kind of complexity LangGraph's graph structure is designed to handle cleanly.

## LangGraph — Core Concepts

*Workflows, Nodes, Edges, State and Reducers — with flow diagrams and interview Q&A*

---

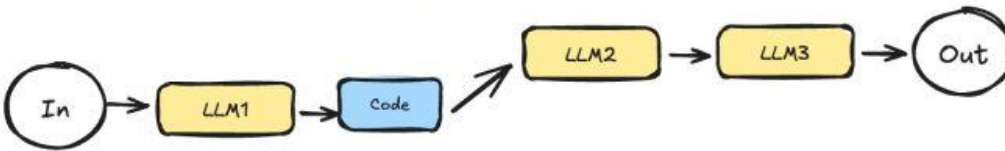
### Introduction

LangGraph is an orchestration framework for building intelligent, stateful, multi-step LLM workflows. It takes any workflow you want to build and represents it as a graph — a set of tasks (nodes) connected by rules that decide what runs next (edges).

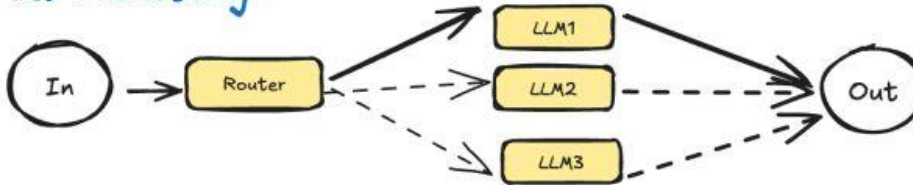
This document covers two things every LangGraph learner needs to know first: the five common LLM workflow patterns you will see again and again, and the core building blocks of LangGraph itself — nodes, edges, state, and reducers. Each topic includes a plain-English explanation and a flow diagram, followed by simple interview-style questions at the end.

### Part 1 — Five Common LLM Workflows

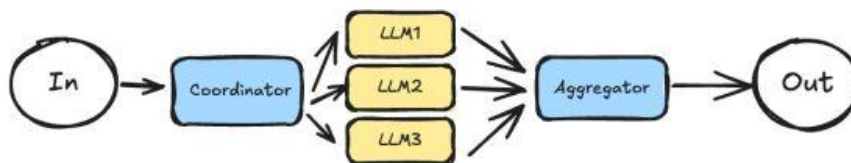
## 1. Prompt Chaining



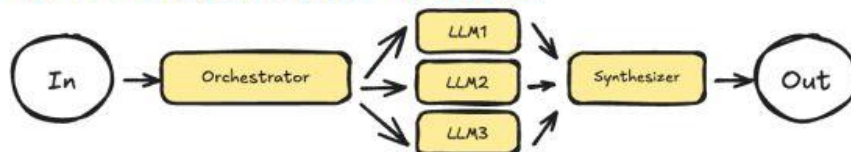
## 2. Routing



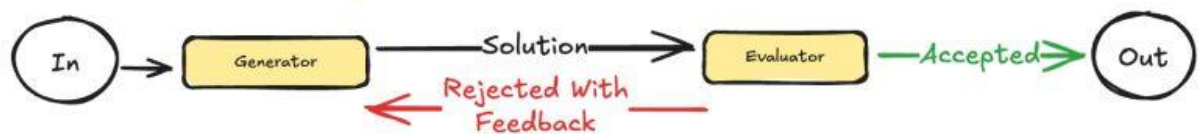
## 3. Parallelization



## 4. Orchestrator-Worker



## 5. Evaluator-Optimizer



### 1. Prompt Chaining

**Definition:** Calling an LLM multiple times in sequence, where the output of one call becomes the input of the next.

Easy explanation: Instead of asking one LLM to complete a big task in a single shot, you break the task into smaller ordered steps and pass the result forward — like an assembly line. You can also add a check between steps to catch mistakes early, before they move further down the chain.

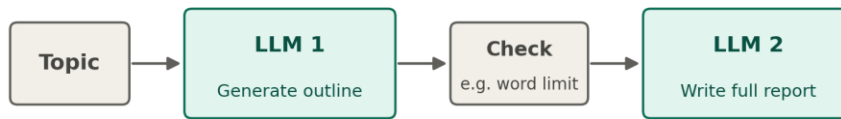


Figure 1: Topic goes to LLM 1 to create an outline, passes through a check, then LLM 2 expands it into a full report.

**Real example:** User gives a topic, LLM 1 creates an outline, then LLM 2 expands that outline into a complete report.

**Use when:** The task is complex and naturally breaks into ordered sub-steps, and you want to verify quality at each stage.

## 2. Routing

**Definition:** An LLM examines the input and decides which specialized path or model should handle it.

Easy explanation: Think of it like a receptionist at the front desk. A query comes in, the router LLM reads it and decides where it should go next — instead of forcing one single LLM to handle every type of request equally well.

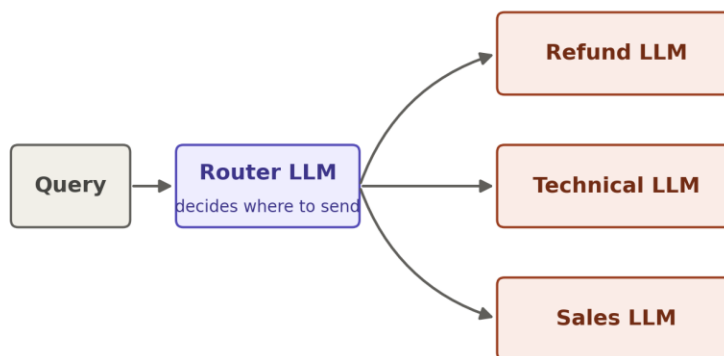


Figure 2: A router LLM reads the query and sends it to the refund, technical, or sales specialist.

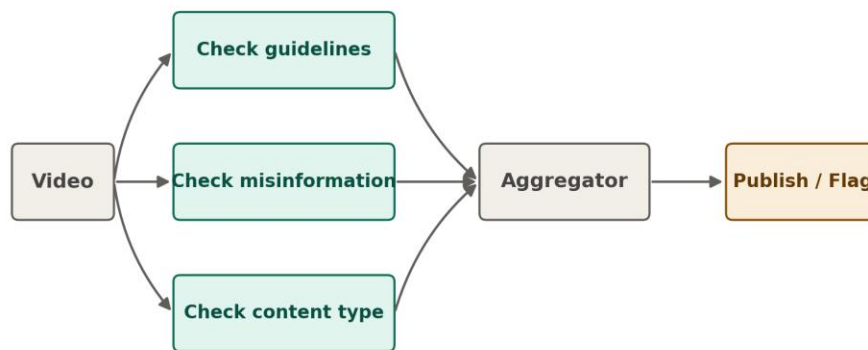
**Real example:** A customer support chatbot routes queries to refund, technical, or sales specialist LLMs depending on what the customer is asking.

**Use when:** You have clearly different types of requests that each need different handling or expertise.

### 3. Parallelization

**Definition:** Break one task into multiple fixed subtasks, run them all at the same time, then merge (aggregate) the results.

Easy explanation: Instead of checking things one after another, you check them all at once — since they don't depend on each other — and then combine the results at the end. This saves time compared to doing everything sequentially.



*Figure 3: A video is checked for guidelines, misinformation, and content type all at once, then aggregated into a decision.*

**Real example:** YouTube-style content moderation — checking community guidelines, misinformation, and inappropriate content simultaneously, then deciding together whether to publish.

**Use when:** Subtasks are independent of each other and you already know in advance exactly what they are.

### 4. Orchestrator–Worker

**Definition:** Similar to parallelization, but the subtasks are not fixed in advance — an orchestrator LLM decides dynamically what each worker should do, based on the input.

Easy explanation: In parallelization, you already know the fixed set of checks you need. Here, you don't know in advance what work is needed — a manager LLM looks at the input first and then assigns different jobs to different workers on the fly.

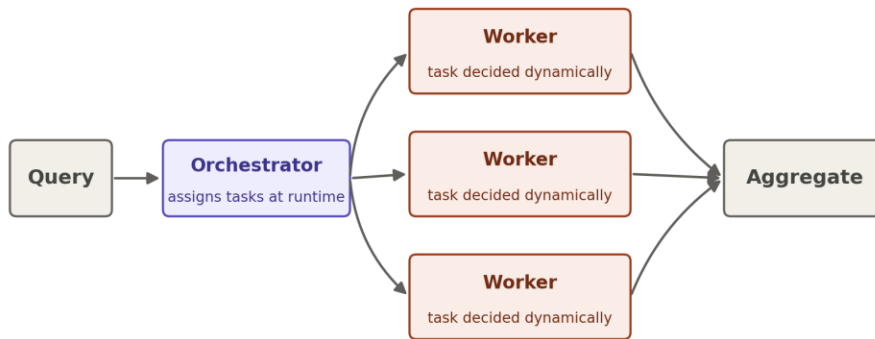


Figure 4: The orchestrator reads the query and assigns a different task to each worker at runtime, then aggregates the results.

**Real example:** A research assistant — if the query is scientific, the orchestrator assigns a worker to search Google Scholar; if it's political or social, it assigns a worker to search Google News.

**Use when:** The nature or number of subtasks varies with the input and can't be hardcoded ahead of time.

**Key difference from Parallelization:** Parallelization = fixed subtasks known upfront. Orchestrator-Worker = subtasks decided dynamically at runtime.

## 5. Evaluator–Optimizer

**Definition:** A generator LLM produces a solution, an evaluator LLM checks it against criteria and either accepts it or sends feedback — this loops until the evaluator is satisfied.

Easy explanation: Like a writer and an editor. The writer drafts something, the editor reviews it and either says it's good or asks for changes with specific feedback. The writer revises, and this repeats until the editor approves. Useful for creative or quality-sensitive tasks that are hard to get right in one attempt.

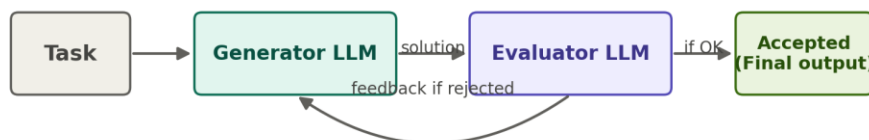


Figure 5: The generator produces a solution, the evaluator checks it, and rejected solutions loop back with feedback.

**Real example:** Drafting an email or blog post — generate a draft, evaluate it, send feedback, regenerate, and repeat until it passes the bar.

**Use when:** Quality can't be guaranteed in one attempt and iterative refinement genuinely helps — writing, poems, essays, or code review.

### Quick Comparison

| Workflow            | Subtasks known ahead?   | Sequence or parallel? | Has a loop?         |
|---------------------|-------------------------|-----------------------|---------------------|
| Prompt Chaining     | N/A - single path       | Sequence              | No                  |
| Routing             | N/A - one path chosen   | Single branch chosen  | No                  |
| Parallelization     | Yes, fixed              | Parallel              | No                  |
| Orchestrator-Worker | No, decided dynamically | Parallel              | No                  |
| Evaluator-Optimizer | N/A                     | Sequence              | Yes, until accepted |

## Part 2 — Core Concepts of LangGraph

### Nodes and Edges

**Node - Definition:** A node represents one single task in the workflow. Behind the scenes, a node is literally just a Python function.

**Edge - Definition:** An edge tells the graph what should run next after a node finishes executing. It defines the flow of execution.

Easy explanation: If nodes are the stops, edges are the roads connecting them, telling traffic which way to go next.

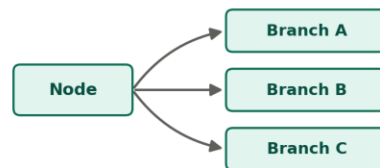
### Types of Edges

- Sequential — one node runs, then the next, in a straight line.
- Parallel — multiple nodes run at the same time after one node finishes.
- Conditional (branching) — based on a condition, the flow goes one way or another.
- Loop — the flow goes back to a previous node, repeating a cycle.

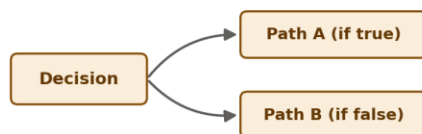
**Sequential edge - one after another**



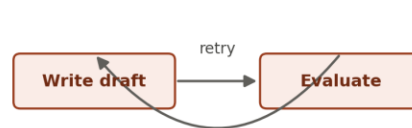
**Parallel edges - many run at the same time**



**Conditional edge - branches on a condition**



**Loop edge - goes back to repeat a step**



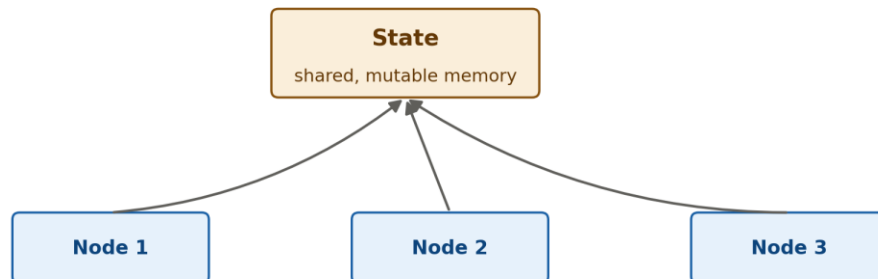
*Figure 6: The four edge types — sequential, parallel, conditional, and loop.*

### State

**Definition:** State is a shared, mutable memory that flows through the entire graph. It holds all the data being passed between nodes as the graph runs.



Easy explanation: Think of state as a shared notebook that every node can read from and write into. As execution moves from node to node, this notebook keeps getting updated, and everyone downstream sees the latest version.



*Figure 7: Every node reads from and writes to the same shared state.*

### Key properties

- Shared — every node in the graph can access it.
- Mutable — nodes can change or update it.
- Implemented in code as a TypedDict (a typed Python dictionary), sometimes a Pydantic object.

**Real example:** In a UPSC essay evaluator app, the state might hold the essay text, clarity score, depth score, language score, and overall score — all filled in and updated as the workflow progresses.

### Reducers

**Definition:** A reducer defines how an update from a node is applied to the state — whether it replaces, appends to, or merges with the existing value. Each key in state can have its own reducer.

Easy explanation: When a node updates the state, does the new value overwrite the old one, or get added on top of it? That rule is decided by the reducer. Get this wrong and you can accidentally lose important data.

### Replace reducer



Old message is lost, so the LLM cannot recall the name

### Append reducer



Nothing is lost, so the full conversation history is kept

Figure 8: With a replace reducer the earlier message is lost; with an append reducer, nothing is lost.

### Rule of thumb

- Use replace when only the latest value matters (e.g. a running score/total).
- Use append/add when history matters (e.g. chat messages, multiple draft versions).

**Real example:** In a chatbot, replacing the messages key every time makes the bot forget earlier context. Using an add reducer keeps the full conversation so the bot can refer back to it.

## LangGraph Core Concepts

Chapter 4 Notes — Definitions, Simple Explanations, Code & Diagrams

### Introduction

This chapter covers the foundational ideas you need before writing any LangGraph code: what LangGraph actually is, the common patterns LLM workflows fall into, and the four building blocks that every graph is made of — nodes, edges, state, and reducers — plus how LangGraph executes a graph behind the scenes.

Each topic below includes a simple definition, a plain-English explanation, and either a diagram or the actual code syntax used to implement it.

### 1. What is LangGraph?

**Definition:** LangGraph is an orchestration framework for building intelligent, stateful, and multi-step LLM workflows. It converts any LLM workflow into a graph structure where each node represents a task, and edges define the flow of execution between tasks.

Easy explanation: You hand LangGraph a workflow, and it turns that workflow into a flowchart-like graph. Once you give the first node some input and trigger the graph, every node runs automatically in the right order until the workflow is complete.

#### Key Characteristics

| Component  | Description                                              |
|------------|----------------------------------------------------------|
| Nodes      | Each node represents a single task in your workflow.     |
| Edges      | Connections between nodes that define execution order.   |
| Task Types | Can include LLM calls, tool calls, decision making, etc. |

#### Advanced Features

| Feature      | Description                                                |
|--------------|------------------------------------------------------------|
| Parallelism  | Execute multiple tasks simultaneously.                     |
| Loops        | Return to previous nodes and execute in cycles.            |
| Branching    | Conditional execution based on certain conditions.         |
| Memory       | Record and maintain conversation history and task outputs. |
| Resumability | Resume from a specific point if the workflow breaks down.  |

**Why use LangGraph:** building complex AI agents, production-grade AI applications, workflows that need state management, and systems that need human-in-the-loop interactions.

## 2. LLM Workflows

**Definition:** A workflow is a series of tasks that you execute in order to achieve a goal: Workflow = Series of Tasks -> Executed in Order -> Goal Achievement.

**Example:** an automated hiring workflow — Create JD -> Post Job -> Shortlisting -> Interview -> Onboarding. Each step must run in the correct order to reach the final goal of hiring a candidate.

**What makes it an LLM workflow:** an LLM Workflow is a workflow where multiple tasks depend on LLMs for their execution. In the hiring example, JD creation needs an LLM to generate the description, shortlisting may use an LLM for resume screening, and interviews could use LLM-based assessment.

### Types of Tasks Inside a Workflow

- Prompting — sending queries to the LLM.
- Reasoning — logical decision making.
- Tool Calling — invoking external tools or APIs.
- Memory Access — retrieving stored information.
- Decision Making — conditional logic execution.

### Workflow Structure Types

| Type     | Description / Use Case                               |
|----------|------------------------------------------------------|
| Linear   | Sequential execution A -> B -> C — simple pipelines. |
| Parallel | Simultaneous execution — independent subtasks.       |
| Branched | Conditional paths — decision-based flows.            |
| Looped   | Iterative execution — refinement processes.          |

### 3. Common LLM Workflow Patterns

While every application has its own unique workflow, five patterns show up again and again. Knowing them well makes it much easier to design new workflows.

#### 3.1 Prompt Chaining

**Definition:** calling the LLM multiple times in sequence, where each call builds on the previous one, often with a validation gate/check in between.



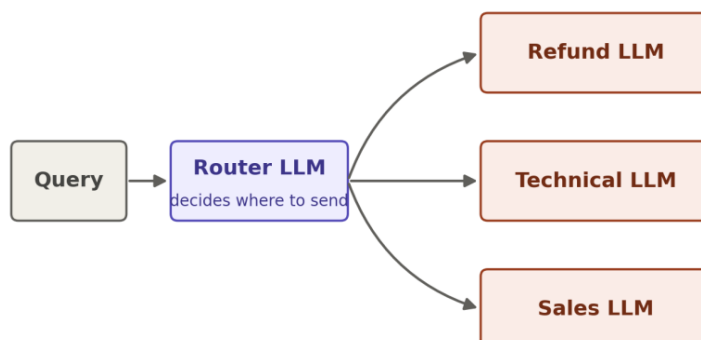
*Prompt chaining: topic -> outline -> check -> full report.*

**Real example — Report Generation:** instead of generating a report directly from a topic (which is often low quality), you first ask an LLM to generate an outline, then a second LLM call expands that outline into the full report, with a check such as "word count < 5000?" before finalizing.

**When to use it:** complex tasks that need to be broken into subtasks, tasks needing intermediate validation, or multi-stage content generation (outline to draft to final).

#### 3.2 Routing

**Definition:** analyzing a task and deciding which specialized handler should process it.



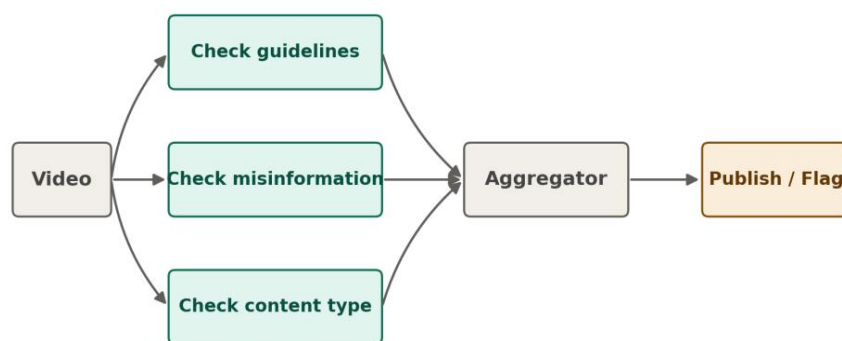
*A router LLM sends the query to the right specialist handler.*

**Real example — Customer Support Chatbot:** a router LLM reads the incoming query and sends "How do I configure the API?" to the Tech Support LLM, "I want to return my order" to the Refund Handler LLM, and "What pricing plans do you offer?" to the Sales Support LLM.

**When to use it:** multi-domain applications with specialized handlers, or whenever different queries genuinely need different expertise.

### 3.3 Parallelization

**Definition:** breaking a task into multiple subtasks that execute simultaneously, then merging their results into a final output.



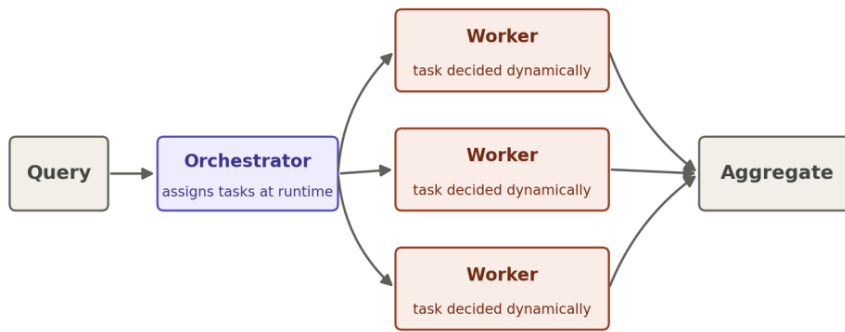
*Three independent checks run at the same time, then get aggregated.*

**Real example — YouTube Content Moderation:** a video's transcript is checked at the same time for community-guideline violations, misinformation, and sexual content. Since none of the three checks depends on the others' results, they can run in parallel; the aggregator publishes the video only if all checks pass, otherwise it flags it.

**When to use it:** tasks with independent subtasks, when speed matters, or multi-criteria evaluation scenarios.

### 3.4 Orchestrator Workers

**Definition:** similar to parallelization, but the subtasks are dynamically determined based on the input rather than being predefined.



*The orchestrator decides each worker's job at runtime, then a synthesizer combines results.*

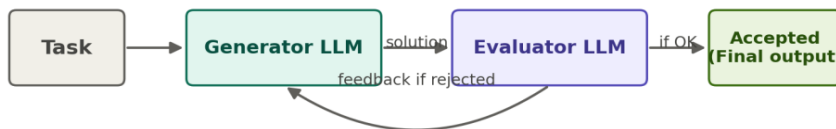
**Key difference from Parallelization:** in parallelization the subtasks are fixed and known beforehand; in orchestrator workers, an orchestrator LLM decides at runtime what each worker should do, so the roles are flexible instead of fixed.

**Real example — Research Assistant:** if the query is a scientific term, the orchestrator assigns a worker to search Google Scholar; if it's a political incident, a worker searches Google News; if it's a technical topic, a worker searches documentation. The nature of the subtasks changes with the input.

**When to use it:** when the task's nature varies with input, research/analysis applications, or multi-source information gathering.

### 3.5 Evaluator Optimizer

**Definition:** an iterative pattern where a Generator LLM creates solutions and an Evaluator LLM provides feedback, looping until the output meets quality criteria.



*The generator and evaluator loop until the evaluator accepts the solution.*

Easy explanation: this pattern mimics how human writers actually work — write a first draft, review it, find issues, write an improved draft, and repeat until it's good enough. A poet or writer never produces perfect content in one attempt.

## Real-World Examples

| Use Case                  | Generator Role / Evaluator Criteria                                               |
|---------------------------|-----------------------------------------------------------------------------------|
| Email Drafting            | Generator creates email content; evaluator checks tone, clarity, professionalism. |
| Blog Writing              | Generator writes blog posts; evaluator checks SEO, engagement, accuracy.          |
| Code Generation           | Generator writes code; evaluator checks correctness, efficiency, style.           |
| Poetry / Creative Writing | Generator creates content; evaluator checks artistic quality, originality.        |

**When to use it:** creative content generation, quality-critical outputs needing refinement, subjective quality criteria, or whenever first-attempt perfection is unlikely.



## 4. Graphs, Nodes, and Edges

This is the most fundamental concept in LangGraph. When LangGraph processes a workflow, it converts it as follows: LLM Workflow -> Graph Representation -> Nodes + Edges.

### Nodes

**Definition:** nodes represent individual tasks in your workflow. Behind the scenes, a node is simply a Python function that executes a single task (an LLM call, a tool call, or a decision).

```
# A node is simply a Python function
def generate_topic(state):
    # Task: Generate a topic
    topic = llm.generate("Create a UPSC essay topic")
    state["topic"] = topic
    return state
```

### Edges

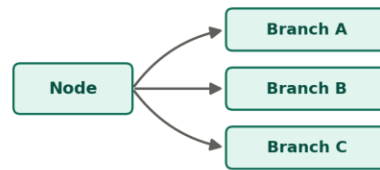
**Definition:** edges connect nodes and define the flow of execution — they tell the graph what should run, and when.

| Edge Type   | Visual Representation                 |
|-------------|---------------------------------------|
| Sequential  | One after another: A -> B -> C        |
| Parallel    | Multiple simultaneous: A -> [B, C, D] |
| Conditional | Based on a condition: A -> {B or C}   |
| Loop        | Back to a previous node: A -> B -> A  |

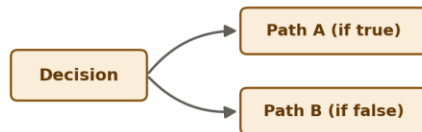
**Sequential edge - one after another**



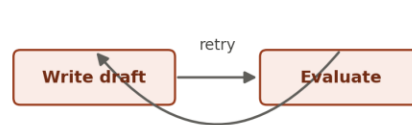
**Parallel edges - many run at the same time**



**Conditional edge - branches on a condition**



**Loop edge - goes back to repeat a step**



*The four edge types: sequential, parallel, conditional, and loop.*

### Worked Example — UPSC Essay Practice Platform

Workflow: generate a topic for the student, collect the essay, evaluate it on multiple criteria (clarity, depth, language — each out of 5, total out of 15, pass threshold 10), aggregate scores, give a pass/fail result, provide feedback if failed, and allow a retry.

**Nodes tell us WHAT to do:** Generate Topic, Write Essay, and Evaluate Clarity are each one task (one Python function).

**Edges tell us WHEN to do it:** after Generate Topic, execute Write Essay; after Write Essay, execute all three evaluators in parallel; after the evaluators, execute Final Evaluation.

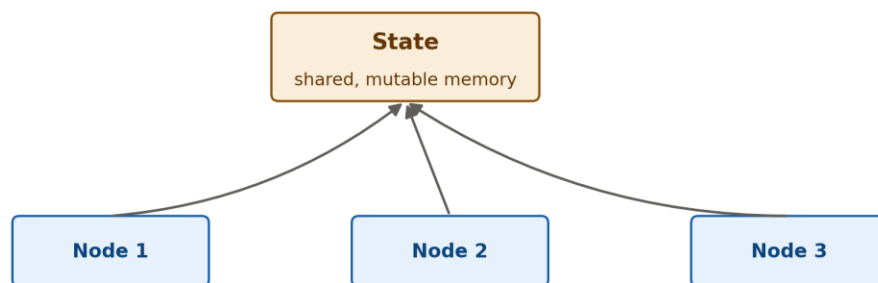
| Flow Type   | Example in This Workflow                                   |
|-------------|------------------------------------------------------------|
| Sequential  | Topic -> Essay: one follows another.                       |
| Parallel    | Essay -> [Clarity, Depth, Language]: all execute together. |
| Conditional | Evaluation -> Success OR Fail: branch based on condition.  |
| Loop        | Fail -> Retry -> Essay: return to an earlier node.         |

## 5. State Management

**Definition:** State is a shared memory that flows through your workflow — it holds all the data being passed between nodes as your graph runs. It is the collection of data points your workflow needs throughout execution, and it evolves over time as the workflow progresses.

### Characteristics of State

| Property  | Description                           |
|-----------|---------------------------------------|
| Shared    | Accessible by all nodes in the graph. |
| Mutable   | Can be modified by any node.          |
| Evolving  | Changes as execution progresses.      |
| Essential | Required for workflow execution.      |



*Every node reads from and writes to the same shared state.*

**How state works:** a node receives the current state as input, executes its task, updates the relevant parts of state, and the updated state is passed on to the next node(s).

### Code Example — UPSC Essay State

```
from typing import TypedDict, Annotated
from operator import add

class EssayState(TypedDict):
    essay_text: str          # The essay content
    topic: str               # Essay topic
    depth_score: int         # Score for depth (0-5)
    language_score: int      # Score for language (0-5)
    clarity_score: int       # Score for clarity (0-5)
    total_score: int         # Combined score
    feedback: Annotated[list[str], add] # List of feedback comments
    evaluation_round: int    # Which attempt number
```

As the workflow runs, this same object gets filled in step by step — starting empty, then topic gets set, then essay\_text gets set after submission, then all three scores and total\_score get set after evaluation. Nothing is recreated; the same state object just evolves.

**Implementation:** State is typically implemented as a Python TypedDict (as above), or as a Pydantic BaseModel when you need stronger validation.

## 6. Reducers

**Definition:** Reducers define how updates are applied to each state value — whether a new value replaces the old one, gets appended to a list, or gets merged in.

### The Problem Reducers Solve

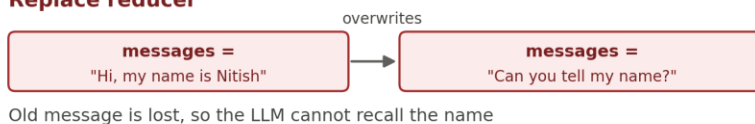
Because state is shared and mutable by every node, the default behavior is that a new value simply replaces the old one. In a simple math workflow (Node 1 sets `first_num=5`, `second_num=6`; Node 2 computes `result=11`; Node 3 computes `result=22`), that's exactly what you want — Node 3 replacing Node 2's result is correct.

But in a chatbot, the same default behavior breaks things: Human says "Hi, my name is Nitish" (message = that text); LLM replies "How can I help you?" (message gets replaced); Human then asks "Can you tell me my name?" — but the LLM can no longer answer, because the message containing the name was overwritten.

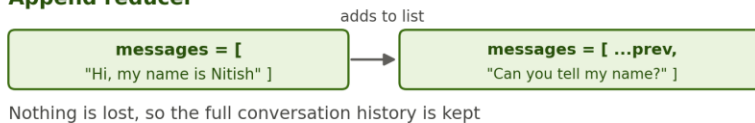
### The Reducer Solution

| Update Policy     | Behavior / Use Case                                                |
|-------------------|--------------------------------------------------------------------|
| Replace (default) | New value replaces old value — for simple values that just change. |
| Add / Append      | New value is added to a list — for message history, logs.          |
| Merge             | Old and new are combined — for dictionary-style updates.           |

#### Replace reducer



#### Append reducer



*Replace loses the earlier message; an add/append reducer keeps full history.*

### Implementing an Add Reducer

```
from typing import TypedDict, Annotated
from operator import add

class ChatState(TypedDict):
    messages: Annotated[list[str], add]    # ADD instead of replace
```

With this reducer, messages accumulate instead of being overwritten:

```
# Message 1: "Hi, my name is Nitish"
messages = ["Hi, my name is Nitish"]

# Message 2: "How can I help you?"
messages = ["Hi, my name is Nitish", "How can I help you?"]

# Message 3: "Can you tell me my name?"
messages = ["Hi, my name is Nitish", "How can I help you?", "Can you tell me my name?"]
```

**Essay platform example:** the same idea preserves every essay attempt (`essay_text: Annotated[list[str], add]`) so a student can see their first, second, and third drafts side by side instead of only the latest one overwriting the rest.

**Key takeaway:** each key in state can have its own reducer — replace, add, merge, or a fully custom function. Reducers are essential for parallel workflows (where multiple nodes update state at once) and for any workflow where history needs to be preserved.

## 7. LangGraph Execution Model

Understanding how LangGraph executes a graph behind the scenes helps with designing and debugging complex applications. This execution model is inspired by Google Pregel, a system designed for large-scale graph processing used across many Google products.

### Phase 1 — Graph Definition

You define three things: the state schema, the nodes, and the edges.

```
from langgraph.graph import StateGraph

# Define state
class MyState(TypedDict):
    value: str

# Create graph
graph = StateGraph(MyState)

# Add nodes
graph.add_node("node1", function1)
graph.add_node("node2", function2)

# Add edges
graph.add_edge("node1", "node2")
```

### Phase 2 — Compilation

Compilation validates the structure of the graph you defined.

```
# Compile the graph
compiled_graph = graph.compile()
```

What compilation checks: no orphaned nodes (disconnected from the graph), valid edge connections, a proper state schema, and no other structural inconsistencies.

### Phase 3 — Invocation

Execution starts by providing an initial state to the entry node.

```
# Run the graph
result = compiled_graph.invoke(initial_state)
```

What happens: the initial state is passed to the first node, that node activates (its function executes), and execution begins.

## Phase 4 — Super-Steps Execution

**Definition:** a Super-Step is one round of execution that may include one or more parallel steps.

Why "super-step" instead of just "step"? Because the graph structure allows parallel execution — a single super-step might contain just 1 node running sequentially, or 3 nodes running together in parallel, or any number of nodes executing at once.

### Message Passing

Message Passing is how state moves between nodes via edges: a node executes its function, updates part of the state (a partial update), the updated state passes through the edge to the next node, and that next node receives the state and activates.

### Termination Conditions

Workflow execution stops when: no active nodes remain, no messages are being passed through any edge, or an end node is reached.

### Automatic Orchestration

Important: you never manually call each node one by one. Once you invoke the graph, nodes automatically execute in the correct order, state automatically passes between nodes, parallel nodes automatically run together, and the framework handles all of the orchestration for you.

### Execution Model — Quick Reference

| Concept          | Description                                   |
|------------------|-----------------------------------------------|
| Graph Definition | Define nodes, edges, state.                   |
| Compilation      | Validate graph structure.                     |
| Invocation       | Start execution with initial state.           |
| Super-Step       | One round of (possibly parallel) execution.   |
| Message Passing  | State transfer via edges.                     |
| Activation       | A node becomes active when it receives state. |



## 8. Summary and Key Takeaways

| Aspect | Description                                                  |
|--------|--------------------------------------------------------------|
| What   | Orchestration framework for LLM workflows.                   |
| How    | Represents workflows as graphs with nodes and edges.         |
| Why    | Enables complex, stateful, production-grade AI applications. |

### Workflow Patterns — Quick Reference

| Pattern              | Use When -> Key Feature                                                  |
|----------------------|--------------------------------------------------------------------------|
| Prompt Chaining      | Complex tasks needing breakdown -> sequential LLM calls with validation. |
| Routing              | Different query types need different handlers -> decision-making router. |
| Parallelization      | Independent subtasks -> simultaneous execution.                          |
| Orchestrator Workers | Dynamic subtask assignment -> runtime task determination.                |
| Evaluator Optimizer  | Quality refinement needed -> iterative improvement loop.                 |

### State Management — Quick Reference

| Concept       | Purpose -> Implementation                               |
|---------------|---------------------------------------------------------|
| State         | Shared data across nodes -> implemented as a TypedDict. |
| Reducer       | Controls update behavior -> Annotated[type, add].       |
| Accessibility | All nodes can read/write -> built-in feature.           |
| Evolution     | State changes over execution -> automatic propagation.  |

### Execution Model — Step by Step

- Define -> State, Nodes, Edges
- Compile -> Validate structure
- Invoke -> Start with initial state

- Execute -> Super-steps with message passing
- Complete -> No active nodes, no messages

### Best Practices

- Start with workflow design before coding.
- Identify state requirements early.
- Choose the appropriate reducer for each state key.
- Use the right pattern for your use case.
- Validate graph structure through compilation.
- Test each node independently before integration.

## Part 3 — Interview Questions (Simple, Not Bookish)

These are phrased the way an interviewer would actually ask, not textbook definitions — useful for quick revision before an interview.

### On the 5 Workflows

**Q1. If someone asks you "what's the difference between parallelization and orchestrator-worker," how do you answer in one line?**

In parallelization, you already know upfront what the 3-4 subtasks are. In orchestrator-worker, an LLM figures out the subtasks on the fly based on what the input actually needs.

**Q2. When would you NOT use prompt chaining?**

When the task is simple enough to be done in a single LLM call — chaining adds latency and cost, so only use it when breaking the task down genuinely improves quality or lets you add checks in between.

**Q3. Give a real-world use case for routing, other than customer support.**

A content-generation app where the router decides if a user wants a blog, a tweet, or an email, and sends it to a model or prompt tuned for that format.

**Q4. Why does evaluator-optimizer need two separate LLMs instead of just one LLM checking its own work?**

A single LLM re-reading its own output tends to just agree with itself. A separate evaluator with clear criteria acts like an independent reviewer, giving more honest, structured feedback — closer to how a human editor works.

**Q5. Can workflows be combined? Give an example.**

Yes — e.g. an orchestrator-worker setup where each worker itself uses prompt chaining internally, or a routing step followed by an evaluator-optimizer loop for the chosen path.

### On LangGraph Core Concepts

**Q1. In your own words, what is a node in LangGraph?**

It's just a Python function that does one task — could be calling an LLM, calling a tool, or making a decision.

**Q2. What's the actual difference between a normal edge and a conditional edge?**

A normal edge always goes to the same next node. A conditional edge checks something (like a score or a flag) and decides which of several possible next nodes to go to.

**Q3. Why is state needed at all — why can't each node just work independently?**

Because nodes often need information generated by earlier nodes (like the essay text or previous scores). State is the shared memory that carries that information forward so later nodes aren't starting from scratch.

**Q4. What real bug can happen if you don't think about reducers properly?**

You can silently lose data. For example, in a chatbot, if every new message replaces the old one instead of appending, the bot forgets everything except the last message — it can't recall earlier context like the user's name.

**Q5. If you were designing a system where a user submits an essay multiple times and wants to see improvement, which reducer would you pick for the essay field, and why?**

An append reducer — you want to keep every version of the essay (v1, v2, v3) so the user can track their progress, instead of the newest version wiping out the older ones.

**Q6. What's the difference between state being mutable and just being a normal Python variable?**

It's still just a Python `dict` or `TypedDict` — **mutable** here specifically means every node in the graph has access to update it, and those changes are visible to whichever node runs next. It's about being shared across the whole graph, not just about being editable in Python.

**Q7. How would you explain graph, node, and edges to someone who has never used LangGraph, in 10 seconds?**

You take your workflow, break it into small tasks (nodes), and draw arrows between them showing what runs next (edges) — LangGraph just executes that drawing for you automatically.

## LangGraph — Chapter 5

### Sequential Workflows in LangGraph

*Simple-language notes • Theory • Flow Diagrams • Code Examples • Interview Q&A*

Prepared from CampusX "Agentic AI using LangGraph" course notes

## 1. What This Chapter Is About

This chapter is the first hands-on coding chapter in the LangGraph course. Until now everything was theory (Generative AI vs Agentic AI, LangChain vs LangGraph, core concepts). Here we finally write code and build our first working LangGraph workflows — starting with the simplest possible pattern: the Sequential Workflow.

Think of it as "Hello World" for LangGraph. Once you understand how to connect a few steps in a straight line, every other pattern (branching, looping, parallel, agents) is just a variation of this same basic idea.

### 1.1 Setting Up Your Environment (in plain words)

Before writing any workflow, set up a clean Python environment:

```
# 1. Create a virtual environment
python -m venv myenv

# 2. Activate it (Windows)
myenv\Scripts\activate

# 3. Install the required libraries
pip install langgraph langchain langchain-openai python-dotenv

# 4. Quick import test
from langgraph.graph import StateGraph
```

#### *What this code does:*

- `python -m venv myenv` creates an isolated folder with its own Python interpreter, so this project's packages never clash with other projects on your machine.
- `myenv\Scripts\activate` switches your terminal to use that isolated environment (on Mac/Linux it's `source myenv/bin/activate`).
- `pip install ...` downloads the four libraries the whole chapter depends on, in one line.
- The final import line is just a sanity check — if it runs with no error, LangGraph is installed correctly and you're ready to code.

Why these libraries?

- `langgraph` — the actual framework that lets you build the graph/workflow.
- `langchain` — gives you ready-made LLM building blocks (chat models, prompts, loaders).
- `langchain-openai` — connects LangChain to OpenAI's models.
- `python-dotenv` — safely loads your API keys from a `.env` file instead of hardcoding them.

Simple way to remember it: LangChain gives you the Lego bricks (LLM calls, tools, prompts). LangGraph gives you the board on which you arrange those bricks into a flow.

## 2. What Is a Sequential Workflow? (Theory, Simple Explanation)

A Sequential Workflow is the easiest type of workflow you can build. In plain English: you do Step 1, then Step 2, then Step 3 — one after another, in a straight line, with no decisions and no forks in the road.

It's exactly like a recipe: chop vegetables → boil water → cook rice. You can't cook the rice before boiling the water, and there's no "if" condition — you always do all steps in the same order.

### Key characteristics

- Linear task connection — each step follows directly from the one before it.
- Single path — there is only one possible route through the workflow, no branching.
- Predictable — you always know exactly what happens next.

#### General shape of a Sequential Workflow

**Task 1 → Task 2 → Task 3 → ... → END**

## 2.1 The Four Building Blocks You Already Know

Every LangGraph workflow — sequential or not — is made of the same four pieces. Here's what each one really means in simple terms:

| Concept     | Simple meaning                                                                     | Code equivalent                                     |
|-------------|------------------------------------------------------------------------------------|-----------------------------------------------------|
| State       | A shared "notebook" that gets passed from step to step and carries all the data.   | <code>class MyState(TypedDict): ...</code>          |
| Node        | A single step — just a normal Python function that reads the state and updates it. | <code>def my_node(state): ... return state</code>   |
| Edge        | An arrow that says "after this step, go to that step."                             | <code>graph.add_edge('a', 'b')</code>               |
| START / END | Special markers for where the workflow begins and finishes.                        | <code>from langgraph.graph import START, END</code> |

Node function rule (important): a node always takes the state as input and always returns the (updated) state. It should only touch the fields it actually needs to change — this is called a partial update.

## 2.2 The Standard 6-Step Recipe to Build Any Sequential Workflow

1. Define the State — a TypedDict listing every field of data that will flow through the workflow.
2. Create the Graph object — `StateGraph(YourState)`.
3. Write Node functions — plain Python functions, one per step.
4. Register the nodes — `graph.add_node('name', function)`.

5. Connect them with edges — `graph.add_edge(...)`, starting from `START` and ending at `END`.
6. Compile and run — `workflow = graph.compile()` then `workflow.invoke(initial_state)`.

You will see this exact 6-step recipe repeat in all three examples below — only the state fields and the node logic change.

## 3. Example 1 — BMI Calculator (No LLM)

This is the very first LangGraph workflow in the course, and on purpose it does NOT use an LLM. The goal is to learn LangGraph's syntax without the added complexity of AI calls. It's just plain Python math wrapped inside LangGraph's structure.

### 3.1 What it does

Given a person's weight (kg) and height (m), the workflow calculates their BMI and then labels it into a category (Underweight / Normal / Overweight / Obese).

**BMI Calculator Workflow**  
`START → calculate_bmi → label_bmi → END`

(A simpler version with just one node also appears in the notes: `START → calculate_bmi → END`, before the category-labelling node is added.)

### 3.2 State Definition

```
from typing import TypedDict

class BMIState(TypedDict):
    weight_kg: float    # Weight in kilograms
    height_m: float     # Height in meters
    bmi: float          # Calculated BMI value
    category: str        # BMI category
```

#### *What this code does:*

- `TypedDict` (from Python's `typing` module) lets you define a dictionary shape where every key has a declared type — it behaves like a normal dict at runtime, but your editor/type-checker can warn you if you forget a field or use the wrong type.
- `BMIState` is just a contract: "any state object in this workflow will have these four keys." It doesn't hold any actual values yet — it's a blueprint, not an instance.
- You'll notice `category` isn't filled in until the second node runs — that's fine. Fields can be added to state progressively as nodes execute.

### 3.3 Node Functions



```

def calculate_bmi(state: BMIState) -> BMIState:
    weight = state['weight_kg']
    height = state['height_m']
    bmi = weight / (height ** 2)
    state['bmi'] = round(bmi, 2)
    return state

def label_bmi(state: BMIState) -> BMIState:
    bmi = state['bmi']
    if bmi < 18.5:
        state['category'] = 'Underweight'
    elif 18.5 <= bmi < 25:
        state['category'] = 'Normal'
    elif 25 <= bmi < 30:
        state['category'] = 'Overweight'
    else:
        state['category'] = 'Obese'
    return state

```

#### *What this code does:*

- `calculate_bmi(state)` pulls `weight_kg` and `height_m` out of the incoming state, applies the standard BMI formula  $\text{weight} \div \text{height}^2$ , rounds it to 2 decimal places, writes it back into `state['bmi']`, and returns the whole state object.
- `label_bmi(state)` reads the `bmi` value that the previous node just wrote, runs it through a simple if/elif/else ladder against the standard medical BMI ranges, and stores the matching label in `state['category']`.
- Notice `label_bmi` never touches `weight_kg` or `height_m` — it only reads `bmi` and writes `category`. This is the "partial update" idea: each node minds its own business and leaves the rest of the state untouched.
- Because `label_bmi` depends on `state['bmi']` existing, it **MUST** run after `calculate_bmi` — this dependency is exactly what the edges below enforce.

### 3.4 Build, Compile and Run

```

from langgraph.graph import StateGraph, START, END

graph = StateGraph(BMIState)

graph.add_node('calculate_bmi', calculate_bmi)
graph.add_node('label_bmi', label_bmi)

graph.add_edge(START, 'calculate_bmi')
graph.add_edge('calculate_bmi', 'label_bmi')
graph.add_edge('label_bmi', END)

workflow = graph.compile()

initial_state = {'weight_kg': 80, 'height_m': 1.73}
final_state = workflow.invoke(initial_state)
print(final_state)
# {'weight_kg': 80, 'height_m': 1.73, 'bmi': 26.73, 'category': 'Overweight'}

```

#### What this code does:

- StateGraph(BMIState) creates an empty graph object and tells it "every state flowing through you will follow the BMIState shape."
- add\_node('calculate\_bmi', calculate\_bmi) registers the function under a string name. The name is what you'll reference when wiring edges — it doesn't have to match the function name, but keeping them identical avoids confusion.
- The three add\_edge(...) calls draw the actual arrows: START → calculate\_bmi, calculate\_bmi → label\_bmi, label\_bmi → END. Together they describe the exact path in section 3.1's diagram.
- graph.compile() checks the graph is well-formed (every node reachable, no dangling edges) and hands back a workflow object you can actually run.
- workflow.invoke(initial\_state) kicks off execution: it feeds initial\_state into calculate\_bmi, takes that output and feeds it into label\_bmi, and returns the final combined state — now containing weight\_kg, height\_m, bmi and category all together.

### 3.5 Complete, Runnable Script (Full Version)

The notes also give the entire BMI example as one self-contained, production-style script — with docstrings, a reusable run\_bmi\_workflow() wrapper function, and multiple test cases. This is worth studying because it shows how the same six pieces from above get organized into a real Python file rather than scattered notebook cells.

```
# BMI Calculator Workflow - LangGraph Tutorial
# Complete implementation following the tutorial

from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# Step 1: Define State
class BMIState(TypedDict):
    """
    State structure for BMI calculation workflow
    Contains all data that flows through the workflow
    """
    weight_kg: float    # Weight in kilograms
    height_m: float     # Height in meters
    bmi: float          # Calculated BMI value
    category: str       # BMI category classification

# Step 2: Define Node Functions

def calculate_bmi(state: BMIState) -> BMIState:
    """
    Calculate BMI using the formula: weight / height^2

    Args:
        state: Current state containing weight and height
    Returns:
        Updated state with calculated BMI value
    """
    # Extract values from state
    weight = state['weight_kg']
```

```

    height = state['height_m']

    # Calculate BMI using standard formula
    bmi = weight / (height ** 2)

    # Update state with calculated value (rounded to 2 decimal places)
    state['bmi'] = round(bmi, 2)

    # Return updated state
    return state

def label_bmi(state: BMIState) -> BMIState:
    """
    Categorize BMI value into standard health categories

    Args:
        state: Current state containing calculated BMI
    Returns:
        Updated state with BMI category
    """
    # Extract BMI value from state
    bmi = state['bmi']

    # Categorize BMI according to standard ranges
    if bmi < 18.5:
        state['category'] = 'Underweight'
    elif 18.5 <= bmi < 25:
        state['category'] = 'Normal'
    elif 25 <= bmi < 30:
        state['category'] = 'Overweight'
    else:
        state['category'] = 'Obese'

    return state

# Step 3: Build the Graph

# Create graph object with state type
graph = StateGraph(BMIState)

# Add nodes to the graph (mapping node names to functions)
graph.add_node('calculate_bmi', calculate_bmi)
graph.add_node('label_bmi', label_bmi)

# Add edges to define the workflow sequence
graph.add_edge(START, 'calculate_bmi')
graph.add_edge('calculate_bmi', 'label_bmi')
graph.add_edge('label_bmi', END)

# Step 4: Compile the graph
workflow = graph.compile()

# Step 5: Execute the workflow

def run_bmi_workflow(weight_kg: float, height_m: float) -> dict:
    """
    """

```

```

Execute the BMI calculation workflow

Args:
    weight_kg: Weight in kilograms
    height_m: Height in meters
Returns:
    Complete state with BMI and category
"""
# Define initial state
initial_state = {
    'weight_kg': weight_kg,
    'height_m': height_m
}

# Execute workflow
final_state = workflow.invoke(initial_state)

return final_state

# Example usage
if __name__ == '__main__':
    # Test the workflow with sample data
    result = run_bmi_workflow(weight_kg=80, height_m=1.73)

    print('BMI Calculation Results:')
    print(f"Weight: {result['weight_kg']} kg")
    print(f"Height: {result['height_m']} m")
    print(f"BMI: {result['bmi']}")
    print(f"Category: {result['category']}")

    # Expected output:
    # Weight: 80 kg
    # Height: 1.73 m
    # BMI: 26.73
    # Category: Overweight

    # For Jupyter Notebook visualization (uncomment if using Jupyter):
    # from IPython.display import Image
    # Image(workflow.get_graph().draw_mermaid_png())

    # Alternative test cases
    test_cases = [
        {'weight_kg': 60, 'height_m': 1.75, 'expected_category': 'Normal'},
        {'weight_kg': 50, 'height_m': 1.70, 'expected_category': 'Underweight'},
        {'weight_kg': 90, 'height_m': 1.65, 'expected_category': 'Obese'},
    ]

    print('\nTesting multiple cases:')
    for i, case in enumerate(test_cases, 1):
        result = run_bmi_workflow(case['weight_kg'], case['height_m'])
        print(f"Test {i}: BMI={result['bmi']}, Category={result['category']}")

```

#### What this code does:

- This is the exact same graph as before — the only difference is organization. The state, nodes, graph-building and compile step are now written once at module load time (when the file is imported/run), instead of being re-typed in every notebook cell.

- `run_bmi_workflow(weight_kg, height_m)` is a thin wrapper around `workflow.invoke()`. Wrapping it in a normal function means other code can call `run_bmi_workflow(80, 1.73)` without needing to know anything about LangGraph, state dictionaries, or the word 'invoke' — good practice for handing a workflow to teammates.
- `if __name__ == '__main__':` is standard Python — the code inside only runs when you execute this file directly (`python bmi_workflow.py`), not when another file imports functions from it.
- The `test_cases` list plus the for loop is a lightweight way to sanity-check the workflow against several known inputs/outputs at once, instead of manually re-running it for each case.
- Docstrings (the triple-quoted `""" ... """` text) don't affect how the code runs — they're documentation that tools like `help()` and IDEs display, which matters once you're writing code others will maintain.

### 3.6 Bonus: Visualizing the Graph (Jupyter only)

```
from IPython.display import Image
Image(workflow.get_graph().draw_mermaid_png())
```

#### *What this code does:*

- `workflow.get_graph()` asks the compiled workflow for its internal graph structure (nodes + edges).
- `draw_mermaid_png()` renders that structure as an actual PNG image using Mermaid diagram syntax under the hood.
- `Image(...)` then displays that PNG inline in a Jupyter cell — this only works inside Jupyter/IPython, not in a plain `.py` script run from the terminal.

This prints an actual picture of your graph — great for debugging and for showing your workflow to teammates.

## 4. Example 2 — Simple LLM Q&A Workflow

Now we bring in an actual LLM call. This is the simplest possible LLM workflow: one node, ask a question, get an answer.

**Simple LLM Q&A Workflow**  
**START → llm\_qa → END**

### 4.1 State Definition

```
class LLMState(TypedDict):
    question: str
    answer: str
```

#### *What this code does:*

- Only two fields this time: question is the input the user provides, answer is what the single node will fill in. Much smaller state than the BMI example because there's only one processing step.

## 4.2 Node Function

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI()

def llm_qa(state: LLMState) -> LLMState:
    question = state['question']
    prompt = f'Answer the following question {question}'
    answer = model.invoke(prompt).content
    state['answer'] = answer
    return state
```

### What this code does:

- `load_dotenv()` reads your `.env` file (where your `OPENAI_API_KEY` should live) and loads it into environment variables — this is why you never hardcode the API key directly in your script.
- `model = ChatOpenAI()` creates one reusable chat-model client object. Notice it's created once, outside the node function — you don't want to recreate the connection on every single call.
- Inside `llm_qa`: the question is pulled from state, wrapped into a plain text prompt using an f-string, and sent to the model with `model.invoke(prompt)`.
- `model.invoke(prompt)` returns a full response object (with metadata like token usage) — `.content` pulls out just the actual text reply.
- The reply is stored back into `state['answer']` and the whole state is returned, exactly like the BMI nodes did.

## 4.3 Build and Run

```
graph = StateGraph(LLMState)
graph.add_node('llm_qa', llm_qa)
graph.add_edge(START, 'llm_qa')
graph.add_edge('llm_qa', END)
workflow = graph.compile()

initial_state = {'question': 'How far is moon from the earth?'}
final_state = workflow.invoke(initial_state)
print(final_state['answer'])
# 'The average distance between the Moon and Earth is about
# 384,400 kilometers (238,855 miles).'
```

### What this code does:

- Same build pattern as before, just with one node instead of two: `START` → `llm_qa` → `END`.
- `initial_state` only needs to supply question — answer doesn't exist yet; it gets added by the node during execution.

- `final_state['answer']` after `invoke()` contains the model's reply, now permanently attached to that state object for as long as you keep a reference to it.

#### 4.4 So... why not just call the LLM directly?

A fair question — `model.invoke('How far is moon from the earth?').content` would give the exact same answer with far less code. The point of this example isn't efficiency; it's to show LangGraph's basic wiring (`state` → `node` → `edges` → `compile` → `invoke`) using the smallest possible LLM example before things get more complex. LangGraph starts paying off once you have multiple LLM calls that depend on each other — which is exactly the next example.

### 5. Example 3 — Prompt Chaining (Blog Generator)

Prompt Chaining means calling the LLM multiple times in sequence, where the output of one call becomes the input to the next. Instead of asking the LLM to "write a blog" in one giant, unfocused prompt, we break it into two smaller, focused steps: first generate an outline, then use that outline to write the full blog.

This mirrors how a human writer actually works — plan first, then write — and it consistently produces better structured output than a single mega-prompt.

**Prompt Chaining Workflow**  
**START → create\_outline → create\_blog → END**

#### 5.1 State Definition

```
class BlogState(TypedDict):
    title: str      # input topic
    outline: str    # intermediate result
    content: str    # final blog output
```

Notice the state has three fields that map exactly to `input` → `intermediate` → `final output`. This is a common and very useful pattern: design your state around the stages of your pipeline.

#### 5.2 Node Functions

```
def create_outline(state: BlogState) -> BlogState:
    title = state['title']
    prompt = f'Generate a detailed outline for a blog on the topic - {title}'
    outline = model.invoke(prompt).content
    state['outline'] = outline
    return state

def create_blog(state: BlogState) -> BlogState:
    title = state['title']
    outline = state['outline']
```

```

prompt = f'Write a detailed blog on the title - {title} using the following
outline \n {outline}'
content = model.invoke(prompt).content
state['content'] = content
return state

```

#### *What this code does:*

- `create_outline` reads only title (the raw topic) and asks the LLM to produce a structured outline. That outline is written into `state['outline']` — nothing else in state is touched.
- `create_blog` is where the chaining actually happens: it reads BOTH title and the outline that the previous node just generated, and builds a new prompt that explicitly includes that outline as context.
- Because `create_blog`'s prompt embeds outline directly, the second LLM call is grounded in the first call's output — this is the literal mechanism of "prompt chaining." Without LangGraph's state, you'd have to manually pass the outline variable between functions yourself.
- The final content is written to `state['content']`, completing the input → intermediate → final chain defined in the state.

### 5.3 Build and Run

```

graph = StateGraph(BlogState)
graph.add_node('create_outline', create_outline)
graph.add_node('create_blog', create_blog)

graph.add_edge(START, 'create_outline')
graph.add_edge('create_outline', 'create_blog')
graph.add_edge('create_blog', END)

workflow = graph.compile()

initial_state = {'title': 'Rise of AI in India'}
final_state = workflow.invoke(initial_state)

print(final_state['title'])    # original topic
print(final_state['outline'])  # generated outline
print(final_state['content'])  # final blog

```

#### *What this code does:*

- The edges enforce order: `create_outline` must finish (and write `state['outline']`) before `create_blog` runs, because `create_blog` depends on that value. Getting the edge order wrong here would crash the second node with a missing-key error.
- `initial_state` only needs 'title' — both outline and content are populated automatically as the workflow runs, node by node.
- After `invoke()` finishes, all three fields are readable from `final_state` — this is the "every intermediate result stays accessible" advantage discussed in section 5.4.

### 5.4 Why This Beats a Plain LangChain Call



| Aspect       | Traditional Chain                  | LangGraph                                                           |
|--------------|------------------------------------|---------------------------------------------------------------------|
| State access | Only the final output is available | Every intermediate value (title, outline, content) stays accessible |
| Flexibility  | Limited control mid-pipeline       | Full control over state at every step                               |
| Debugging    | Hard to see where it went wrong    | Easy — inspect the state after any node                             |

## 5.5 Homework Given in the Course (good practice exercise)

Add a third node called `evaluate_blog` that takes the outline and the generated blog, asks the LLM to rate it, and stores an integer score in the state.

```
class BlogState(TypedDict):
    title: str
    outline: str
    content: str
    score: int  # new field

def evaluate_blog(state: BlogState) -> BlogState:
    outline = state['outline']
    content = state['content']
    prompt = f'Based on this outline, rate my blog and generate an integer score:
    Outline: {outline}
    Blog: {content}'
    score = model.invoke(prompt).content
    state['score'] = int(score)
    return state

graph.add_node('evaluate_blog', evaluate_blog)
graph.add_edge('create_blog', 'evaluate_blog')
graph.add_edge('evaluate_blog', END)
```

### What this code does:

- Adding `score: int` to `BlogState` is step one for any new capability — the state must declare a field before any node can write to it.
- `evaluate_blog` follows the exact same shape as the other two nodes: read what you need from state (here, outline and content), build a prompt, call the model, write the result back.
- `int(score)` converts the LLM's text reply into an actual integer — a reminder that `model.invoke(...).content` is always a string, so any numeric or structured value needs manual conversion (and in real code, some error handling in case the LLM doesn't return a clean number).
- Rewiring the edges — `create_blog` now points to `evaluate_blog` instead of straight to `END`, and `evaluate_blog` points to `END` — extends the straight-line chain from 2 steps to 3 without changing anything else about how the graph works.

Try building this yourself — it's a great way to check you actually understood the pattern rather than just copy-pasting it.

## 6. Chapter Cheat Sheet

### Three progressive examples in this chapter

- 1. BMI Calculator : START → calculate\_bmi → label\_bmi → END
- 2. LLM Q&A : START → llm\_qa → END
- 3. Prompt Chaining : START → create\_outline → create\_blog → END

### 6.1 The 6-step pattern, one last time

- 7. Define State (TypedDict)
- 8. Create graph = StateGraph(YourState)
- 9. Write node functions (input state → output state)
- 10. graph.add\_node(name, function)
- 11. graph.add\_edge(...) — chain START → nodes → END
- 12. workflow = graph.compile() then workflow.invoke(initial\_state)

### 6.2 Key takeaways

- A Sequential Workflow = straight-line execution, no branching.
- State is the single source of truth that flows through every node.
- Nodes only update the parts of state they're responsible for.
- LangGraph keeps every intermediate result accessible — traditional chains don't.
- Start simple, add complexity gradually — always use type hints.

## 7. Interview Questions & Answers

These are framed the way an interviewer would actually ask them — practical and scenario-based, not textbook definitions. Answer in your own words using the ideas below.

### **Q1. What exactly is a "Sequential Workflow" in LangGraph, and how would you explain it to someone who's never used LangGraph?**

It's the simplest workflow shape — steps run one after another in a fixed order, like a checklist. There's no decision-making, no loops, no parallel paths. If you can draw your process as a single straight arrow (Step 1 → Step 2 → Step 3), it's sequential. I'd compare it to following a recipe: you can't skip steps or reorder them.

### **Q2. In the BMI example, why did the course deliberately avoid using an LLM?**

Because mixing a new concept (LangGraph's graph/state/node syntax) with another new concept (LLM prompting) at the same time makes debugging confusing — you can't tell if a bug is in your graph wiring or your prompt. Using plain Python math isolates the LangGraph mechanics so you learn the framework cleanly first.

### **Q3. If I just want to answer one question with an LLM, why would I ever wrap that in a LangGraph workflow instead of calling `model.invoke()` directly?**

For a single call, you wouldn't — direct invocation is simpler and more efficient. LangGraph's value shows up once you have more than one step that depends on shared state: multiple LLM calls, conditional logic, retries, human approval steps, parallel branches. The single-node example exists purely to teach the wiring pattern before those real use-cases arrive.

### **Q4. What is the difference between State, a Node, and an Edge — in your own words, not the textbook definition?**

- State: a shared dictionary-like object (TypedDict) that travels through the whole workflow and holds every piece of data anyone might need.
- Node: one unit of work — just a Python function that reads from state and writes back to it.
- Edge: the wiring — it tells LangGraph which node runs after which. START and END are just special edge anchors marking where the flow begins and finishes.

### **Q5. Why does every node function have to return the state instead of just modifying it in place?**

LangGraph is designed around explicit state updates — a node receives the current state and must return the (partial or full) update, which LangGraph then merges back into the overall state. Returning it explicitly keeps the data flow visible and predictable, which matters a lot once you have branching or parallel nodes writing to state — LangGraph needs to know exactly what each node changed.

### **Q6. What is Prompt Chaining, and when would you actually use it in a real project?**

It's making multiple LLM calls in sequence, where each call's output feeds the next call's input — instead of trying to get everything from one giant prompt. You'd use it any time a task naturally breaks into stages: plan then write (like the outline → blog example), extract then summarize, draft then critique then revise. Splitting the task usually gives more structured, controllable output than one mega-prompt.

### **Q7. The interviewer gives you a task: "summarize a document, then translate the summary to French." How would you design the LangGraph state and nodes?**

State would need at least three fields: document (input), summary (intermediate), translated\_summary (final output) — following the same input → intermediate → final pattern as the blog example. Two nodes: summarize\_doc (reads document, writes summary) and translate\_summary (reads summary, writes translated\_summary). Edges: START → summarize\_doc → translate\_summary → END. This is a direct rename of the prompt-chaining pattern — recognizing that is the real skill being tested.

#### **Q8. What's the actual advantage of LangGraph's state model over a plain LangChain sequential chain?**

In a traditional chain you typically only get the final output — the intermediate steps disappear once the next step consumes them. In LangGraph every intermediate value stays inside the state object and is accessible after the run finishes (final\_state['outline'] as well as final\_state['content']). That makes debugging, auditing, and reusing intermediate results much easier.

#### **Q9. What happens if two nodes in a sequential graph both try to update the same state key?**

In the basic sequential setup shown here, each node runs one after another and simply overwrites that key with its own value — whichever node runs last "wins," since there's no branching or concurrency yet. (Note: LangGraph does support custom reducers to control exactly how updates to a key are combined — that becomes important once you move beyond simple sequential flows into parallel or looping graphs.)

#### **Q10. Why is TypedDict used for State instead of a regular Python dict or a Pydantic model?**

TypedDict gives you type hints for every field (so your editor/IDE and type checkers can catch mistakes) while still behaving like a normal dictionary at runtime — no extra validation overhead. It's a lightweight way to document "what data this workflow expects" without the runtime cost of a validation library, which is ideal for LangGraph state that gets read/written very frequently.

#### **Q11. Walk me through what happens, step by step, when you call workflow.invoke(initial\_state).**

- LangGraph starts at the START node and looks up the edge to find the first real node.
- It calls that node's function with the current state and receives back an update, which gets merged into state.
- It follows the next edge to the next node, repeating the same read-update-merge cycle.
- This continues until it reaches an edge pointing to END.
- The final, fully-updated state object is returned to the caller.

#### **Q12. Your BMI workflow needs to reject negative or zero height/weight values. How would you add that without breaking the sequential pattern?**

Simplest approach: add the validation logic at the top of calculate\_bmi and raise an exception (or set an 'error' field in state) if the input is invalid — no new node required for something this small. If the validation logic gets more complex, add it as its own node (e.g. validate\_input) placed before calculate\_bmi: START → validate\_input → calculate\_bmi → label\_bmi → END. This still stays sequential; validation is just another step in the line.

#### **Q13. Why compile the graph at all — why not just call the node functions directly in order?**

Calling functions manually would work for something this trivial, but compile() is what turns your node/edge definitions into a validated, executable, reusable workflow object. It checks the graph structure is valid (no dangling nodes, valid START/END connections) and gives you a single workflow object you can invoke repeatedly, visualize, and later extend with things like checkpointing/persistence — none of which you'd get from just calling functions in a script.

**Q14. What's a realistic limitation of sequential workflows that would push you toward a more advanced LangGraph pattern?**

Sequential workflows can't make decisions — every run follows the exact same path. The moment your logic needs an "if this, do X, else do Y" (e.g., only translate if the summary is in English, or retry a node if the LLM output fails validation), you need conditional edges / branching, which is the next topic after sequential workflows. Sequential is the foundation; branching, looping, and parallel execution are built on top of the same state/node/edge vocabulary.

**Q15. If you were explaining to a junior developer why LangGraph is useful even though "it's just calling functions in order," what would you say?**

I'd say: yes, for a simple sequential case, it is just functions in order — the value isn't in this simple case, it's in what the same mental model gives you for free later. Once your workflow needs branching, loops, parallel steps, persistence across sessions, human-in-the-loop pauses, or visual debugging, you don't switch frameworks — you just add more nodes and edges to the same graph. Learning sequential workflows first is really about learning the vocabulary (state, node, edge) that scales up to those harder problems.